L09 — Claude Ecosystem and Artifacts

Lecture text for a 90–120 minute practical session

GenAI Basics · Module 09 · 2026-09-08

Purpose of the module

This module turns Claude from “another chatbot” into a practical workbench. Students learn where Claude fits in a GenAI toolchain, how Artifacts change the workflow from conversational answers to reusable outputs, and how the developer-facing building blocks—Messages, tools, files, citations, context, caching and Skills—connect to real product and workplace use cases.

> ***Instructor stance: do not sell Claude as magic. Teach it as an opinionated workspace with useful affordances and real governance obligations.***

Learning outcomes

- Explain the main Claude surfaces: Claude chat/Desktop, Artifacts, Projects/connectors, Claude Code, Developer Platform, cloud deployments and Skills.
- Select the right surface for exploration, reusable content, repository work, application integration or stateful agent workflows.
- Write an artifact request that contains purpose, audience, source data, constraints, interaction model, visual hierarchy and acceptance tests.
- Run a practical artifact iteration loop: build, inspect, test, revise and decide whether to share, export or rebuild.
- Describe how the Messages API changes when tools, files, citations, context management, prompt caching or Skills are added.
- Identify risks around public links, organization sharing, file isolation, tool side effects and untrusted Skills.

1. Claude is an ecosystem, not a single chat box

A beginner usually sees Claude as a text assistant. That is a limited mental model. The useful mental model is: Claude is a set of surfaces that trade off speed, control, governance and integration effort.

Surface	Best fit	Operational caveat
Claude chat / Desktop	Thinking partner, drafting, analysis, first-pass artifacts	Easy to over-trust; user must verify outputs and settings.
Artifacts	Reusable standalone content: dashboards, documents, single-page apps, diagrams, code snippets	The output can look finished before it is tested.
Projects / connectors	Persistent working context, team knowledge, source-connected	Connector permissions and source freshness need governance.

	analysis	
Claude Code	Repository-aware coding and codebase maintenance	Repo state, tool permissions and irreversible edits must be controlled.
Claude API / Messages	Embedding Claude into a product or workflow	You own state, retries, permissions, tool execution, logging and monitoring.
Managed Agents / cloud platforms	Longer-running or enterprise-controlled deployments	Availability, data retention and feature parity vary by platform and contract.
Skills	Reusable workflows and file-generation capabilities	Skills are code/instructions; treat external Skills like dependencies.

2. Why Artifacts matter

Artifacts change the unit of work. In plain chat, the output is usually a message. With Artifacts, the output can be a reusable object: a document, a web page, a dashboard, a diagram or an interactive component. That changes how we prompt and how we evaluate quality. The question is no longer “did the answer sound right?” but “does this artifact solve the task when a real person uses it?”

A good Artifact is substantial, self-contained and worth reusing outside the immediate conversation. That is why an artifact request should read more like a product brief than like a casual prompt.

3. The artifact build loop

1. Frame the job: user, decision, source data, output format and non-goals.
2. Give Claude a compact but explicit artifact brief.
3. Inspect the generated artifact as a user: Can I find the answer? Can I trust the data? Can I take the next action?
4. Run acceptance tests: factual checks, UI checks, data handling checks and edge cases.
5. Revise with targeted diffs, not vague taste feedback.
6. Decide the deployment path: keep private, export, publish publicly or share only inside an organization.

4. Prompt anatomy for useful artifacts

Prompt part	What to specify	Example
Purpose	The decision or workflow the artifact supports	Help a support lead spot stale high-priority tickets.
Audience	Who will use it and what they already know	Team lead, non-technical, wants a 3-minute morning check.
Inputs	Files, columns, assumptions and dirty-data warnings	CSV with id, customer, product, severity, age_days, status, owner.
Output	Artifact type and interaction model	Single-page dashboard with filters, summary KPIs and ticket table.
Constraints	Style, privacy, tools, size, non-goals	No external libraries; do not show customer emails; use readable labels.
Acceptance tests	How we judge success	If any P1 ticket is older than 2 days, it must be visible above the fold.
Revision policy	How to ask for changes	After the first version, list tradeoffs

		before changing layout.

5. Live demo: support triage artifact

The demo uses a small support-ticket dataset. Students first see a weak request produce an attractive but shallow output. Then the instructor uses a stronger artifact brief and acceptance tests. The point is not to worship the tool; the point is to show that a structured brief makes the model easier to inspect, correct and govern.

6. Developer building blocks beneath the surface

A product team using Claude through the API is not “using the same thing as the web app.” It is constructing messages, choosing models, deciding what context to pass, deciding whether to attach files, deciding how to handle tool calls, and deciding what gets logged or shared.

Building block	Beginner explanation	Why engineers care
Messages	The turn-by-turn protocol used to ask Claude for output.	Your application controls state and replay.
Tool use	Claude can request an action; your app or Anthropic infrastructure executes it.	Permission, schema design and side effects are your responsibility.
Files API	Upload once, reference by file id in requests.	Useful for repeated document/data workflows; workspace boundaries matter.
PDF / document input	Claude can work with documents as source material.	Evidence traceability matters when answers influence decisions.
Citations	Claude can return references to source documents.	Reduces “trust me” answers; still requires source quality checks.
Context management	The model reads what fits in the working window.	Long context needs curation; more text can reduce reliability.
Prompt caching	Stable prefixes can be reused to reduce latency and cost.	Cache hits require stable prompt segments; changing tool definitions breaks value.
Skills	Reusable instructions/scripts/resources for recurring work.	A skill is a dependency and should have ownership and tests.

7. Governance and safety checks

- Treat public sharing as publication, not “just a link.” A link can be forwarded, indexed or copied.
- Keep sensitive input out of public artifacts. Redact before upload, not after publishing.
- Separate tenants or projects when using workspace-scoped file storage or shared Skills.
- Review third-party Skills like you would review scripts from the internet: read instructions, dependencies and network behavior.
- For tool-using applications, make the application responsible for permissions, audit logs, idempotency and kill switches.
- Measure artifact quality by successful task completion and safe reuse, not by first-impression aesthetics.

8. The practice artifact

Students produce an Artifact Build Brief and a revised artifact for a small operational scenario. The brief is intentionally reusable: later modules can adapt it for open models, agents and Streamlit prototypes.

Key takeaways

- Artifacts are useful because they make output inspectable and reusable; they are not automatically correct.
- A strong artifact prompt is a product brief plus acceptance tests.
- Claude API work is product engineering: state, tools, files, context, cost and governance must be designed.
- Skills turn repeated work into reusable capability, but they need security review and evaluation.
- For enterprise use, the boring controls—ownership, permissions, source notes and auditability—are the difference between demo and deployment.

Primary sources checked

Claude Platform documentation home: https://platform.claude.com/docs/en/home — Developer surfaces: Messages, Managed Agents, cloud platform availability.

Tool use with Claude: https://platform.claude.com/docs/en/agents-and-tools/tool-use/overview — Client tools, server tools, tool_use/tool_result loop, strict tool use, tool cost components.

Prompt caching: https://platform.claude.com/docs/en/build-with-claude/prompt-caching — Automatic/explicit caching, cache hit requirements, metrics, TTL, invalidation caveats.

Files API: https://platform.claude.com/docs/en/build-with-claude/files — Upload once, reference file_id, workspace-level isolation and multi-tenant warning.

PDF support: https://platform.claude.com/docs/en/build-with-claude/pdf-support — PDF input options and platform differences for file_id support.

Context windows: https://platform.claude.com/docs/en/build-with-claude/context-windows — Context as working memory, context rot caveat, compaction guidance, usage counts.

Citations: https://platform.claude.com/docs/en/build-with-claude/citations — Document citation flow and current text-only citation limitation.

Agent Skills overview: https://platform.claude.com/docs/en/agents-and-tools/agent-skills/overview — Cross-surface availability, sharing scope, ZDR/data-retention caveats.

Using Agent Skills with the API: https://platform.claude.com/docs/en/build-with-claude/skills-guide — API integration shape, pre-built vs custom Skills, prerequisites.

Skill authoring best practices: https://platform.claude.com/docs/en/agents-and-tools/agent-skills/best-practices — Concise Skills, lazy loading, metadata-first discovery, test with real usage.

Skills for enterprise: https://platform.claude.com/docs/en/agents-and-tools/agent-skills/enterprise — Governance, vetting, deployment, enterprise risk checklist.

Claude Help: Artifacts: https://support.claude.com/en/articles/9487310-what-are-artifacts-and-how-do-i-use-them — Artifact definition, criteria, examples, enablement and iteration flow.

Claude Help: Publish and share artifacts: https://support.claude.com/en/articles/9547008-publish-and-share-artifacts — Publishing vs sharing semantics and access expectations.